

# 184.686 VU Datenbanksysteme

## Transaktionen

Katja Hose

Institut für Logic and Computation

Sommersemester 2026



Informatics

# Lernziele: Transaktionen und Schedules

## Lernziele

- Das Konzept „Transaktion“ verstehen
- Die ACID-Eigenschaften verstehen
- Serialisierbarkeit verstehen
- Das Konzept von Schedules (Historien) verstehen
- Recoverable Schedules und Cascadeless Schedules verstehen

## Motivation

- Transaktionsgrenzen sind ein wichtiger Teil des Systemdesigns
- Benutzer denken in Transaktionen
- Umgang mit Mehrbenutzersystemen

# Übersicht I

- 1 Transaktionen
  - Eigenschaften
  - Operationen auf Transaktionsebene
  - Umsetzung der ACID-Eigenschaften
- 2 Schedules und Serialisierbarkeit
  - Schedules
  - Conflict Serializability
  - Conflict Graphs (Precedence Graphs)
  - Recoverable Schedules und Cascadeless Schedules
- 3 Mehrbenutzersynchronisation
  - Sperrbasierte Synchronisation
  - Zwei-Phasen-Sperrprotokoll (2PL)
  - Konvertierung von Sperren
  - Erkennung von Deadlocks

# Übersicht II

- Vermeidung von Deadlocks

- 4 Recovery
  - Fehlerklassifikation
  - Datenspeicher
  - Log-Einträge
  - Log-Based Recovery

# Einführung

Beispiel einer typischen Bankanwendung:

- ① Lese den Kontostand von A in die Variable  $a$ :  $read(A, a);$
- ② Reduziere den Kontostand um 50 Euro:  $a := a - 50;$
- ③ Schreibe den neuen Kontostand in die Datenbank:  $write(A, a);$
- ④ Lese den Kontostand von B in die Variable  $b$ :  $read(B, b);$
- ⑤ Erhöhe den Kontostand um 50 Euro:  $b := b + 50;$
- ⑥ Schreibe den neuen Kontostand in die Datenbank:  $write(B, b);$

# Einführung

Beispiel einer typischen Bankanwendung:

- ① Lese den Kontostand von A in die Variable  $a$ :  $read(A, a);$
- ② Reduziere den Kontostand um 50 Euro:  $a := a - 50;$
- ③ Schreibe den neuen Kontostand in die Datenbank:  $write(A, a);$
- ④ Lese den Kontostand von B in die Variable  $b$ :  $read(B, b);$
- ⑤ Erhöhe den Kontostand um 50 Euro:  $b := b + 50;$
- ⑥ Schreibe den neuen Kontostand in die Datenbank:  $write(B, b);$

Was könnte ein Problem verursachen?

# Einführung

Beispiel einer typischen Bankanwendung:

- ① Lese den Kontostand von A in die Variable  $a$ :  $read(A, a);$
- ② Reduziere den Kontostand um 50 Euro:  $a := a - 50;$
- ③ Schreibe den neuen Kontostand in die Datenbank:  $write(A, a);$
- ④ Lese den Kontostand von B in die Variable  $b$ :  $read(B, b);$
- ⑤ Erhöhe den Kontostand um 50 Euro:  $b := b + 50;$
- ⑥ Schreibe den neuen Kontostand in die Datenbank:  $write(B, b);$

- Alle Schritte müssen als Einheit betrachtet werden  
“Alles oder nichts”-Prinzip
- Sobald abgeschlossen, müssen die Änderungen permanent gespeichert werden
- ...

# Was ist eine Transaktion?

Eine **Transaktion** ist eine Bündelung mehrerer Datenbankoperationen, die zusammen eine **logische Einheit** bilden. In einer Transaktion wird auf verschiedene Datenbankeinträge zugegriffen, wobei einige Einträge möglicherweise verändert werden.

Transaktionsgrenzen werden durch den Benutzer  
(bzw. die Softwareanwendung) definiert!



# Eigenschaften von Transaktionen: ACID-Eigenschaften

## Atomicity (Atomarität)

- Entweder es werden alle Operationen einer Transaktion korrekt in der Datenbank umgesetzt oder gar keine.
- Wird typischerweise mit Log-Einträgen implementiert.

# Eigenschaften von Transaktionen: ACID-Eigenschaften

## Atomicity (Atomarität)

- Entweder es werden alle Operationen einer Transaktion korrekt in der Datenbank umgesetzt oder gar keine.
- Wird typischerweise mit Log-Einträgen implementiert.

## Consistency (Konsistenz)

- Die Ausführung einer Transaktion in Isolation erhält den konsistenten Zustand der Datenbank.
- Entsprechend der definierten Constraints, Checks, Assertions
- Auch die Anwendung definiert, was Konsistenz ist:  
z.B. sollten Banküberweisungen kein Geld generieren oder zerstören – die Gesamtsumme sollte vorher und nachher gleich sein.

# Eigenschaften von Transaktionen: ACID-Eigenschaften

## Isolation

- Jede Transaktion hat die DB “für sich allein”.
- Zwischenresultate dürfen für andere Transaktionen nicht sichtbar sein.
- Typischerweise mittels Locks (Sperrern) implementiert.

# Eigenschaften von Transaktionen: ACID-Eigenschaften

## Isolation

- Jede Transaktion hat die DB “für sich allein”.
- Zwischenresultate dürfen für andere Transaktionen nicht sichtbar sein.
- Typischerweise mittels Locks (Sperren) implementiert.

## Durability (Dauerhaftigkeit)

- Änderungen von erfolgreich abgeschlossenen Transaktionen dürfen auch bei Systemfehlern nicht verloren gehen.
- Typischerweise mit Log-Einträgen implementiert.

# Menti

## Frage 3

# Übersicht

- 1 Transaktionen
  - Eigenschaften
  - Operationen auf Transaktionsebene
  - Umsetzung der ACID-Eigenschaften

# Operationen auf Transaktionsebene

## begin of transaction (BOT)

Repräsentiert den Beginn einer Transaktion, d.h., alle Folgebefehle zusammen bilden eine Transaktion.

In SQL        `BEGIN;`

# Operationen auf Transaktionsebene

## begin of transaction (BOT)

Repräsentiert den Beginn einer Transaktion, d.h., alle Folgebefehle zusammen bilden eine Transaktion.

In SQL        `BEGIN;`

## commit

Repräsentiert das Ende einer Transaktion, d.h., alle Änderungen werden festgeschrieben und sind auch für andere sichtbar.

In SQL        `COMMIT;`



# Operationen auf Transaktionsebene

## begin of transaction (BOT)

Repräsentiert den Beginn einer Transaktion, d.h., alle Folgebefehle zusammen bilden eine Transaktion.

In SQL        `BEGIN;`

## commit

Repräsentiert das Ende einer Transaktion, d.h., alle Änderungen werden festgeschrieben und sind auch für andere sichtbar.

In SQL        `COMMIT;`

## rollback oder abort

Führt zu einem “roll back” der Transaktion, d.h., alle Änderungen werden zurückgesetzt/verworfen.

In SQL        `ROLLBACK;`

# Operationen auf Transaktionsebene

## “autocommit” Modus

Jeder Befehl wird in einer eigenen Transaktion ausgeführt.

# Einfache Konsistenzprüfungen (Checks)

```
CREATE TABLE emp(  
  eid      INT          PRIMARY KEY,  
  ename    VARCHAR(30) NOT NULL,  
  salary   INT          NOT NULL CHECK (salary > 0)  
);
```

```
-- primary key violation  
insert into emp values (11, 'Kim', 200);  
-- Not null constraint violation  
insert into emp values (44, NULL, 200);  
-- Check statement violation  
insert into emp values (44, 'Kim', -200);
```

## Einfache Konsistenzprüfungen (Checks)

```
CREATE TABLE emp(  
  eid      INT          PRIMARY KEY,  
  ename    VARCHAR(30) NOT NULL,  
  salary   INT          NOT NULL CHECK (salary > 0)  
);
```

```
-- primary key violation  
insert into emp values (11, 'Kim', 200);  
-- Not null constraint violation  
insert into emp values (44, NULL, 200);  
-- Check statement violation  
insert into emp values (44, 'Kim', -200);
```

- Viele Fehler können vom DBMS erkannt werden!
- Verwenden Sie diese Checks!

# Savepoints

Transaktionen von langer Dauer können zusätzlich Savepoints definieren

```
SAVEPOINT savepoint_name;
```

Definiert einen Punkt/Zustand innerhalb einer Transaktion.

Eine Transaktion kann bis zum Savepoint **teilweise rückgängig gemacht werden**.

# Savepoints

Transaktionen von langer Dauer können zusätzlich Savepoints definieren

```
SAVEPOINT savepoint_name;
```

Definiert einen Punkt/Zustand innerhalb einer Transaktion.

Eine Transaktion kann bis zum Savepoint **teilweise rückgängig gemacht werden**.

```
ROLLBACK TO savepoint_name;
```

Setzt die aktive Transaktion zurück bis zum Savepoints savepoint\_name

# Beispiel

```
BEGIN;  
INSERT INTO tab VALUES...  
SAVEPOINT A;  
INSERT INTO tab VALUES...  
SAVEPOINT B;  
SELECT * FROM tab;  
ROLLBACK TO A;  
SELECT * FROM tab;  
...
```

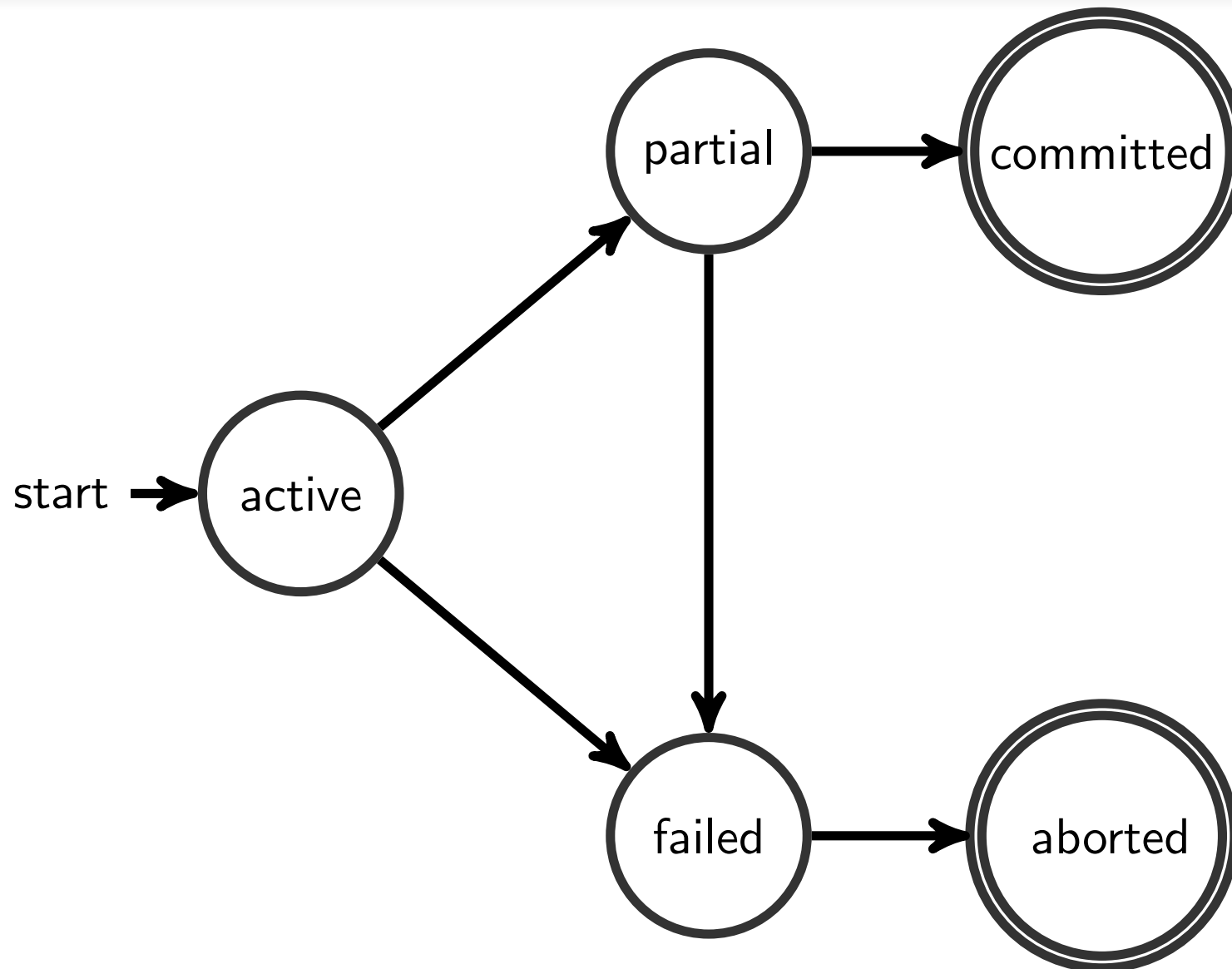
# Abschluss einer Transaktion

Für den Abschluss einer Transaktion gibt es drei Möglichkeiten:

- ① Den **erfolgreichen** Abschluss durch **Commit**
- ② Den **erfolglosen** Abschluss durch ein **Abort**
- ③ Den **erfolglosen** Abschluss durch einen **Fehler**



# Zustandsdiagramm für Transaktionen



# Wie werden Transaktionen vom DBMS realisiert?

Die beiden wichtigsten Komponenten der Transaktionsverwaltung sind

## Mehrbenutzersynchronisation (Isolation)

- Semantische Korrektheit bei Nebenläufigkeit  
Nebenläufigkeit erlaubt einen hohen Durchsatz.
- Serialisierbarkeit
- Schwächere Isolationsstufen (Isolation Levels)

# Wie werden Transaktionen vom DBMS realisiert?

Die beiden wichtigsten Komponenten der Transaktionsverwaltung sind

## Mehrbenutzersynchronisation (Isolation)

- Semantische Korrektheit bei Nebenläufigkeit  
Nebenläufigkeit erlaubt einen hohen Durchsatz.
- Serialisierbarkeit
- Schwächere Isolationsstufen (Isolation Levels)

## Recovery (Atomicity und Durability)

- Zurücksetzen teilweise ausgeführter Transaktionen
- Wiederausführung von Transaktionen nach Ausfällen (Failures)
- Sicherstellen der Persistenz von Änderungen durch Transaktionen

# Menti

Fragen 4–7

# Übersicht

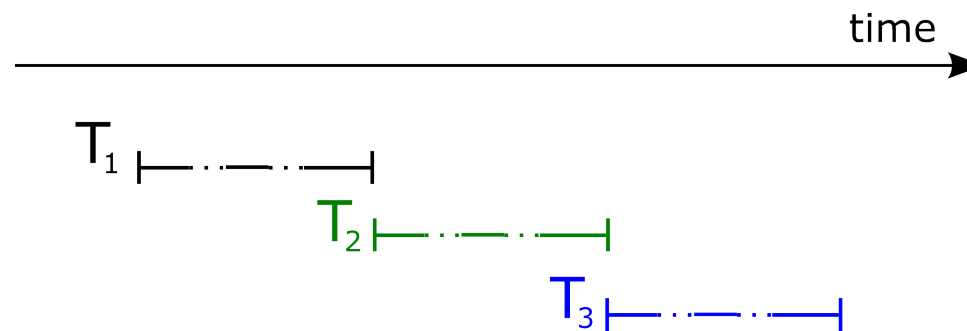
- 2 Schedules und Serialisierbarkeit
  - Schedules
  - Conflict Serializability
  - Conflict Graphs (Precedence Graphs)
  - Recoverable Schedules und Cascadeless Schedules

# Nebenläufigkeit (Parallelität)

Das „I“ in ACID.

Ausführung der drei Transaktionen  $T_1$ ,  $T_2$ , und  $T_3$

(a) im Einzelbetrieb

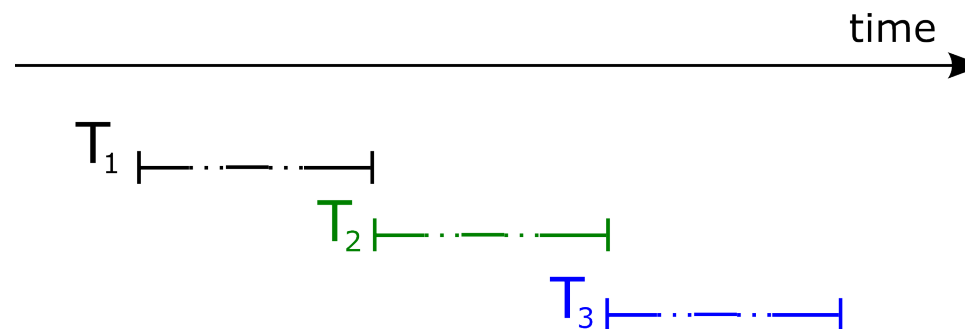


## Nebenläufigkeit (Parallelität)

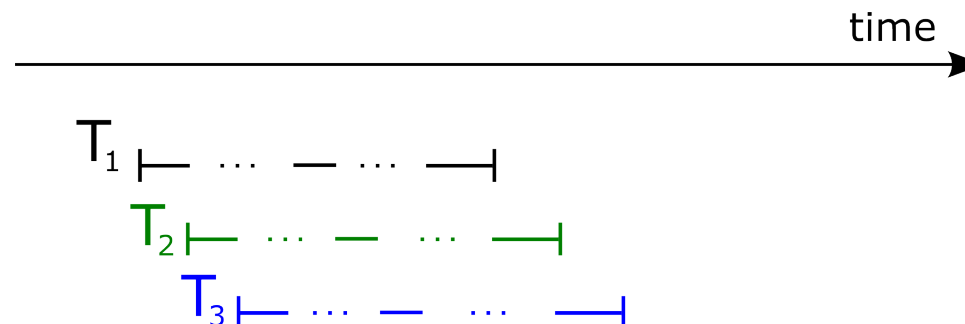
Das „I“ in ACID.

Ausführung der drei Transaktionen  $T_1$ ,  $T_2$ , und  $T_3$

(a) im Einzelbetrieb



(b) im (nebenläufigen) Mehrbenutzerbetrieb mit verzahnter Ausführung



# Probleme bei nebenläufiger Ausführung

Wo liegt hier das Problem?

| Schritt | $T_1$              | $T_2$              |
|---------|--------------------|--------------------|
| 1.      | read(A,a1)         |                    |
| 2.      | $a1 := a1 - 300$   |                    |
| 3.      |                    | read(A,a2)         |
| 4.      |                    | $a2 := a2 * 1.03$  |
| 5.      |                    | <b>write(A,a2)</b> |
| 6.      | <b>write(A,a1)</b> |                    |
| 7.      | read(B,b1)         |                    |
| 8.      | $b1 := b1 + 300$   |                    |
| 9.      | write(B,b1)        |                    |



# Probleme bei nebenläufiger Ausführung

**Lost Updates** (verlorengegangene Änderungen durch Überschreiben)

| Schritt | $T_1$                             | $T_2$                             |
|---------|-----------------------------------|-----------------------------------|
| 1.      | read(A, $a_1$ )                   |                                   |
| 2.      | $a_1 := a_1 - 300$                |                                   |
| 3.      |                                   | read(A, $a_2$ )                   |
| 4.      |                                   | $a_2 := a_2 * 1.03$               |
| 5.      |                                   | <b>write(A, <math>a_2</math>)</b> |
| 6.      | <b>write(A, <math>a_1</math>)</b> |                                   |
| 7.      | read(B, $b_1$ )                   |                                   |
| 8.      | $b_1 := b_1 + 300$                |                                   |
| 9.      | write(B, $b_1$ )                  |                                   |

# Probleme bei nebenläufiger Ausführung

Wo liegt hier das Problem?

| Steps | $T_1$                             | $T_2$                            |
|-------|-----------------------------------|----------------------------------|
| 1.    | read( $A, a_1$ )                  |                                  |
| 2.    | $a_1 := a_1 - 300$                |                                  |
| 3.    | <b>write(<math>A, a_1</math>)</b> |                                  |
| 4.    |                                   | <b>read(<math>A, a_2</math>)</b> |
| 5.    |                                   | $a_2 := a_2 * 1.03$              |
| 6.    |                                   | write( $A, a_2$ )                |
| 7.    | read( $B, b_1$ )                  |                                  |
| 8.    | ...                               |                                  |
| 9.    | abort                             |                                  |

# Probleme bei nebenläufiger Ausführung

**Dirty Read** (Abhängigkeit von nicht freigegebenen Änderungen)

| Steps | $T_1$                             | $T_2$                            |
|-------|-----------------------------------|----------------------------------|
| 1.    | read( $A, a_1$ )                  |                                  |
| 2.    | $a_1 := a_1 - 300$                |                                  |
| 3.    | <b>write(<math>A, a_1</math>)</b> |                                  |
| 4.    |                                   | <b>read(<math>A, a_2</math>)</b> |
| 5.    |                                   | $a_2 := a_2 * 1.03$              |
| 6.    |                                   | write( $A, a_2$ )                |
| 7.    | read( $B, b_1$ )                  |                                  |
| 8.    | ...                               |                                  |
| 9.    | abort                             |                                  |

## Probleme bei nebenläufiger Ausführung

Wo liegt hier das Problem?

| $T_1$                                                                             | $T_2$                                                                                                      |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>update</b> account<br><b>set</b> balance=42000<br><b>where</b> accountID=12345 | <b>select</b> sum(balance)<br><b>from</b> account<br><br><b>select</b> sum(balance)<br><b>from</b> account |

## Probleme bei nebenläufiger Ausführung

**Non-Repeatable Read** (Abhängigkeit von anderen Updates)

| $T_1$                                                                             | $T_2$                                                                                                      |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>update</b> account<br><b>set</b> balance=42000<br><b>where</b> accountID=12345 | <b>select</b> sum(balance)<br><b>from</b> account<br><br><b>select</b> sum(balance)<br><b>from</b> account |

## Probleme bei nebenläufiger Ausführung

Wo liegt hier das Problem?

| $T_1$                                              | $T_2$                                             |
|----------------------------------------------------|---------------------------------------------------|
| <b>insert into account<br/>values (C,1000,...)</b> | <b>select</b> sum(balance)<br><b>from</b> account |
|                                                    | <b>select</b> sum(balance)<br><b>from</b> account |

# Probleme bei nebenläufiger Ausführung

**Phantomproblem** (Abhängigkeit von neuen/gelöschten Tupeln)

| $T_1$                                              | $T_2$                                                                                          |
|----------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>insert into account<br/>values (C,1000,...)</b> | <b>select sum(balance)<br/>from account</b><br><br><b>select sum(balance)<br/>from account</b> |

# Übersicht

- 2 Schedules und Serialisierbarkeit
  - Schedules
  - Conflict Serializability
  - Conflict Graphs (Precedence Graphs)
  - Recoverable Schedules und Cascadeless Schedules



# Nebenläufigkeit und Korrektheit

Zentrales System mit gleichzeitigem Zugriff durch mehrere Benutzer

- Datenbank besteht aus zwei Einträgen: X und Y
- Einziges Kriterium für Korrektheit:  $X = Y$
- Folgende Transaktionen:

$$\begin{array}{ll} T_1 & X \leftarrow X + 1 \\ & Y \leftarrow Y + 1 \end{array} \quad \begin{array}{ll} T_2 & X \leftarrow 2 * X \\ & Y \leftarrow 2 * Y \end{array}$$

- Initial:  $X=10$  und  $Y=10$ .
- $T_1$  gefolgt von  $T_2 \Rightarrow X = 22$  and  $Y = 22$
- $T_2$  gefolgt von  $T_1 \Rightarrow X = 21$  and  $Y = 21$

# Ein Beispiel

| schedule $S_0$ |            |
|----------------|------------|
| $T_1$          | $T_2$      |
|                | read(X, x) |

Wert von X: 10

Wert von Y: 10

# Ein Beispiel

| schedule $S_0$ |                   |
|----------------|-------------------|
| $T_1$          | $T_2$             |
|                | read( $X, x$ )    |
|                | $x \leftarrow 2x$ |

Wert von  $X$ : 10

Wert von  $Y$ : 10

# Ein Beispiel

Wert von X: 20

Wert von Y: 10

| schedule $S_0$ |                                                |
|----------------|------------------------------------------------|
| $T_1$          | $T_2$                                          |
|                | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x) |

# Ein Beispiel

Wert von X: 20

Wert von Y: 10

| schedule $S_0$ |                                                              |
|----------------|--------------------------------------------------------------|
| $T_1$          | $T_2$                                                        |
|                | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y) |

# Ein Beispiel

Wert von X: 20

Wert von Y: 10

| schedule $S_0$ |                   |
|----------------|-------------------|
| $T_1$          | $T_2$             |
|                | read(X, x)        |
|                | $x \leftarrow 2x$ |
|                | write(X, x)       |
|                | read(Y, y)        |
|                | $y \leftarrow 2y$ |

# Ein Beispiel

Wert von X: 20

Wert von Y: 20

| schedule $S_0$ |                   |
|----------------|-------------------|
| $T_1$          | $T_2$             |
|                | read(X, x)        |
|                | $x \leftarrow 2x$ |
|                | write(X, x)       |
|                | read(Y, y)        |
|                | $y \leftarrow 2y$ |
|                | write(Y, y)       |

# Ein Beispiel

Wert von X: 20

Wert von Y: 20

| schedule $S_0$ |                   |
|----------------|-------------------|
| $T_1$          | $T_2$             |
|                | read(X, x)        |
|                | $x \leftarrow 2x$ |
|                | write(X, x)       |
|                | read(Y, y)        |
|                | $y \leftarrow 2y$ |
|                | write(Y, y)       |
| read(X, x)     |                   |



# Ein Beispiel

Wert von X: 20

Wert von Y: 20

| schedule $S_0$                   |                                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                            | $T_2$                                                                                            |
|                                  | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |
| read(X, x)<br>$x \leftarrow x+1$ |                                                                                                  |

# Ein Beispiel

Wert von X: 21

Wert von Y: 20

| schedule $S_0$                                  |                                                                                                  |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                           | $T_2$                                                                                            |
|                                                 | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x) |                                                                                                  |

# Ein Beispiel

Wert von X: 21

Wert von Y: 20

| schedule $S_0$                                                |                                                                                                  |
|---------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                                         | $T_2$                                                                                            |
|                                                               | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br>read(Y, y) |                                                                                                  |

# Ein Beispiel

Wert von X: 21

Wert von Y: 20

| schedule $S_0$                                                                      |                                                                                                  |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                                                               | $T_2$                                                                                            |
|                                                                                     | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow y+1$ |                                                                                                  |

# Ein Beispiel

Wert von X: 21

Wert von Y: 21

| schedule $S_0$                                                                                     |                                                                                                  |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                                                                              | $T_2$                                                                                            |
|                                                                                                    | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) |                                                                                                  |

# Formale Definition eines Schedules

Ein **Schedule** (Historie) ist eine **Sequenz von Operationen** von einer oder mehreren Transaktionen.

Bei nebenläufigen Transaktionen, können die Operationen verzahnt sein.

## Operationen

- $\text{read}(Q, q)$   
Liest den Wert des Datenobjekts  $Q$  und speichert diesen in der lokalen Variable  $q$ .
- $\text{write}(Q, q)$   
Schreibt den Wert der lokalen Variable  $q$  in das Datenobjekt  $Q$ .
- Arithmetische Operationen
- commit
- abort

# Formale Definition eines Schedules

Ein **Schedule** (Historie) ist eine **Sequenz von Operationen** von einer oder mehreren Transaktionen.

Bei nebenläufigen Transaktionen, können die Operationen verzahnt sein.

## Serieller Schedule

Operationen der Transaktionen werden sequenziell und ohne zeitliche Überlappung ausgeführt.

## Nebenläufiger Schedule

Operationen der Transaktionen werden zeitlich überlappend ausgeführt.

# Formale Definition eines Schedules

Ein **Schedule** (Historie) ist eine **Sequenz von Operationen** von einer oder mehreren Transaktionen.

Bei nebenläufigen Transaktionen, können die Operationen verzahnt sein.

## Serieller Schedule

Operationen der Transaktionen werden sequenziell und ohne zeitliche Überlappung ausgeführt.

## Nebenläufiger Schedule

Operationen der Transaktionen werden zeitlich überlappend ausgeführt.

## Gültiger Schedule

Ein Schedule ist gültig (valid), wenn das Resultat der Ausführung “korrekt” ist.



# Menti

Fragen 8–10

## Beispiele:

| schedule $S_0$                                                                                     |                                                                                                  | schedule $S_{0'}$                                                                                      |                                                                                                      | schedule $S_1$                                                                                         |                                                                                                  |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                                                                              | $T_2$                                                                                            | $T_1$                                                                                                  | $T_2$                                                                                                | $T_1$                                                                                                  | $T_2$                                                                                            |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) | read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) | read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |

Sind diese Schedules gültige nebenläufige Schedules, ungültige nebenläufige Schedules, oder serielle Schedules? Initial:  $X=Y=10$ , Korrektheitskriterium:  $X=Y$

## Beispiele:

| schedule $S_0$                                                                                     |                                                                                                  | schedule $S_{0'}$                                                                                      |                                                                                                      | schedule $S_1$                                                                                         |                                                                                                  |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $T_1$                                                                                              | $T_2$                                                                                            | $T_1$                                                                                                  | $T_2$                                                                                                | $T_1$                                                                                                  | $T_2$                                                                                            |
| read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) | read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) | read(X, x)<br>$x \leftarrow x+1$<br>write(X, x)<br><br>read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | read(X, x)<br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$<br>write(Y, y) |

- $X = 21, Y = 21$

- **Serieller Schedule**

- $X = 21, Y = 21$

- **Nebenläufiger Schedule**

- $X = 22, Y = 21$

- **Ungültiger Schedule**

# Korrektheit

## Definition D1

Eine nebenläufige Ausführung von Transaktionen muss die Datenbank in einem konsistentem Zustand hinterlassen.

Geht davon aus, dass jede Transaktion, sofern diese in einem konsistenten Zustand gestartet wurde, die Datenbank in einem konsistentem Zustand hinterlässt.

# Korrektheit

## Definition D1

Eine nebenläufige Ausführung von Transaktionen muss die Datenbank in einem konsistentem Zustand hinterlassen.

Geht davon aus, dass jede Transaktion, sofern diese in einem konsistenten Zustand gestartet wurde, die Datenbank in einem konsistentem Zustand hinterlässt.

## Definition D2

Eine nebenläufige Ausführung von Transaktionen muss ergebnisäquivalent zu einer seriellen Ausführung der Transaktionen sein.

Ergebnisäquivalent heißt, dass die finalen Datenbankzustände identisch sein müssen.

# Menti

Frage 11

# Beispiel

| schedule $S_2$                                  |                                                                                       |
|-------------------------------------------------|---------------------------------------------------------------------------------------|
| $T_3$                                           | $T_4$                                                                                 |
| read(X, x)<br>$x \leftarrow x+1$                |                                                                                       |
| write(X, x)                                     | read(X, x)<br><br>$x \leftarrow 2x$<br>write(X, x)<br>read(Y, y)<br>$y \leftarrow 2y$ |
| read(Y, y)<br>$y \leftarrow y+1$<br>write(Y, y) | write(Y, y)                                                                           |

Initial:  $X = 10$  und  $Y = 10$

$\Rightarrow X = 20$  und  $Y = 20$

- $S_2$  ist nicht ergebnisäquivalent zu einer seriellen Ausführung von  $T_3$ ,  $T_4$
- Aber der finale Datenbankzustand ist konsistent

# Beispiel

| schedule $S_2$                                                                                                                                                       |                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $T_3$                                                                                                                                                                | $T_4$                                                                                                                                                                            |
| $\text{read}(X, x)$<br>$x \leftarrow x+1$<br><br><b><math>\text{write}(X, x)</math></b><br><br><br>$\text{read}(Y, y)$<br>$y \leftarrow y+1$<br>$\text{write}(Y, y)$ | <b><math>\text{read}(X, x)</math></b><br><br>$x \leftarrow 2x$<br><b><math>\text{write}(X, x)</math></b><br>$\text{read}(Y, y)$<br>$y \leftarrow 2y$<br><br>$\text{write}(Y, y)$ |

Initial:  $X = 10$  und  $Y = 10$

$\Rightarrow X = 20$  und  $Y = 20$

- $S_2$  ist nicht ergebnisäquivalent zu einer seriellen Ausführung von  $T_3$ ,  $T_4$
- Aber der finale Datenbankzustand ist konsistent – obwohl es einige Lost Updates gibt.



# Korrektheit von Schedules

Die bessere Wahl ist Definition D2:

Die Ausführungsreihenfolge ist **korrekt**, wenn sie zu einer **seriellen Ausführung ergebnisäquivalent** ist.

# Korrektheit von Schedules

Die bessere Wahl ist Definition D2:

Die Ausführungsreihenfolge ist **korrekt**, wenn sie zu einer **seriellen Ausführung ergebnisäquivalent** ist.

Gegeben ist eine Menge von  $n$  nebenläufigen Transaktionen. Wie können wir effizient auf Korrektheit prüfen?

## Korrektheit von Schedules

Die bessere Wahl ist Definition D2:

Die Ausführungsreihenfolge ist **korrekt**, wenn sie zu einer **seriellen Ausführung ergebnisäquivalent** ist.

Gegeben ist eine Menge von  $n$  nebenläufigen Transaktionen. Wie können wir effizient auf Korrektheit prüfen?

Im Folgenden gehen wir von vereinfachenden Annahmen aus

- **Nur Reads und Writes** werden verwendet, um die Korrektheit festzustellen.
- Diese Annahme ist strenger als Definition D2, da noch weniger Schedules als korrekt gelten.

# Übersicht

- 2 Schedules und Serialisierbarkeit
  - Schedules
  - Conflict Serializability
  - Conflict Graphs (Precedence Graphs)
  - Recoverable Schedules und Cascadeless Schedules

# Eine vierte<sup>1</sup> Definition von Korrektheit: Conflict Serializability (Konfliktserialisierbarkeit)

## Definition (D4<sup>1</sup>)

Ein Schedule ist **conflict serializable** (konfliktserialisierbar) wenn er **konfliktäquivalent** zu einem seriellen Schedule ist.

---

<sup>1</sup>Die dritte Definition (D3) ist Sichtenserialisierbarkeit.

# Mögliche Konflikte zwischen Transaktionen

„Konflikte“ zwischen Paaren von Transaktionen ( $T_1$  und  $T_2$ ) und deren Operationen.

| schedule $S_A$ |            |
|----------------|------------|
| $T_1$          | $T_2$      |
| write(X, x)    | read(X, x) |

**Konflikt**

| schedule $S_B$ |             |
|----------------|-------------|
| $T_1$          | $T_2$       |
| write(X, x)    | write(X, x) |

**Konflikt**

| schedule $S_C$ |            |
|----------------|------------|
| $T_1$          | $T_2$      |
| write(X, x)    | read(X, x) |

**Konflikt**

| schedule $S_D$ |            |
|----------------|------------|
| $T_1$          | $T_2$      |
| read(X, x)     | read(X, x) |

**Kein Konflikt**

## Eine vierte Definition von Korrektheit: Conflict Serializability (Konfliktserialisierbarkeit)

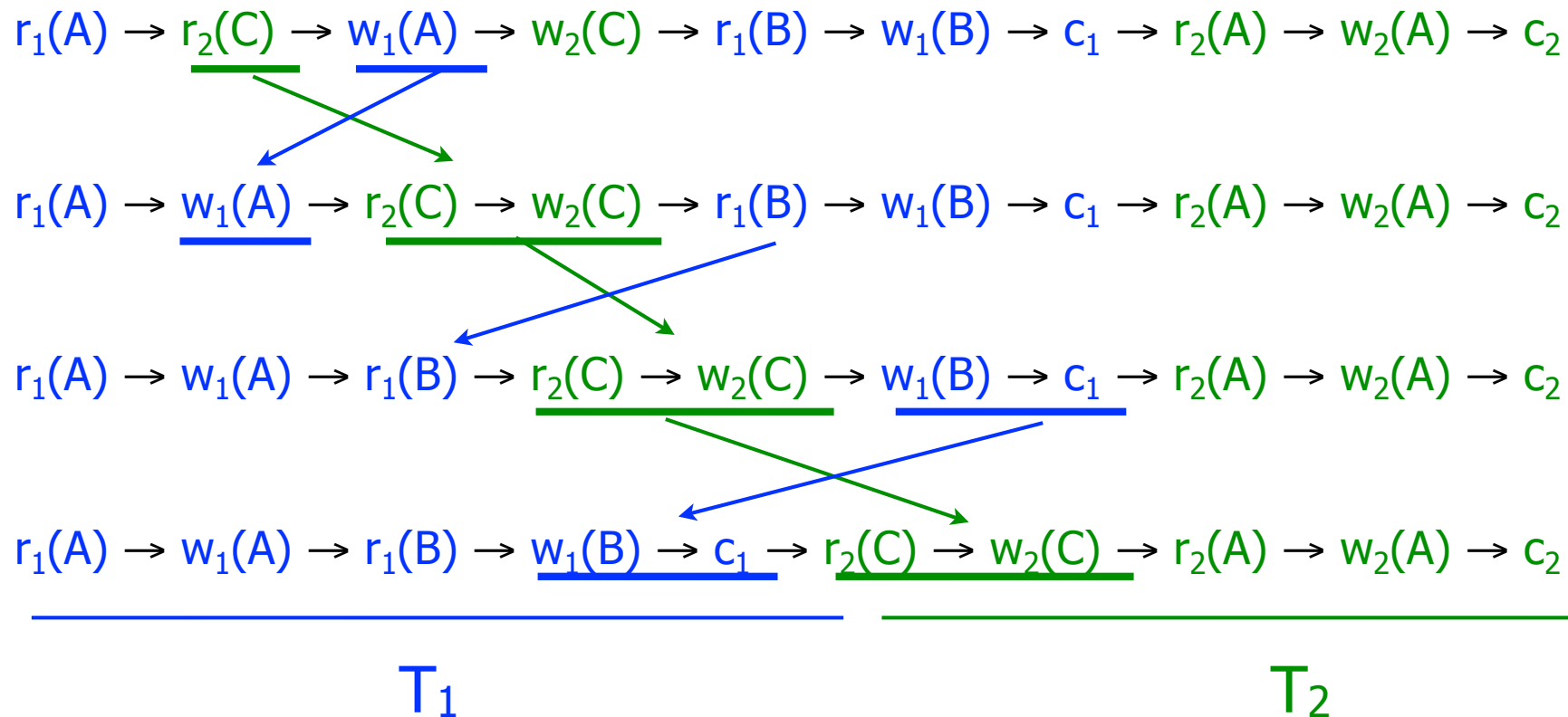
### Definition (D4)

Ein Schedule ist **conflict serializable** wenn er **konfliktäquivalent** zu einem seriellen Schedule ist.

- Seien I und J aufeinanderfolgende Operationen eines Schedules S von mehreren Transaktionen.
- Wenn I und J keinen Konflikt haben, können wir deren Reihenfolge vertauschen, um einen neuen Schedule S' zu erzeugen.
- Die Operationsreihenfolge ist in S und S' gleich, außer I und J.
- S und S' bezeichnen wir als **konfliktäquivalente Schedules**.

# Konfliktäquivalenz von zwei Schedules

Die Transformation zeigt, dass der initiale Schedule konfliktäquivalent zu einem seriellen Schedule ist. Daher ist dieser auch conflict serializable.



c ist die Abkürzung für commit, r (read), w (write)



# Menti

Fragen 12–15

# Conflict serializable oder nicht?

| schedule $S_A$ |             |
|----------------|-------------|
| $T_1$          | $T_2$       |
| read(Y, y)     | read(X, x)  |
|                | write(X, x) |
| write(Y, y)    |             |

# Conflict serializable oder nicht?

| schedule $S_A$ |             |
|----------------|-------------|
| $T_1$          | $T_2$       |
| read(Y, y)     | read(X, x)  |
|                | write(X, x) |
| write(Y, y)    |             |

Conflict serializable

# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Conflict serializable

# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

Conflict serializable

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Nicht conflict serializable

# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

Conflict serializable

| schedule $S_C$ |            |
|----------------|------------|
| $T_5$          | $T_6$      |
| read(X, x)     | read(X, x) |
| write(X, x)    |            |

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Nicht conflict serializable

# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

Conflict serializable

| schedule $S_C$ |            |
|----------------|------------|
| $T_5$          | $T_6$      |
| read(X, x)     | read(X, x) |
| write(X, x)    |            |

Conflict serializable

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Nicht conflict serializable

# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

Conflict serializable

| schedule $S_C$ |            |
|----------------|------------|
| $T_5$          | $T_6$      |
| read(X, x)     | read(X, x) |
| write(X, x)    |            |

Conflict serializable

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Nicht conflict serializable

| schedule $S_D$ |             |
|----------------|-------------|
| $T_7$          | $T_8$       |
| read(X, x)     | write(X, x) |
| write(X, x)    |             |



# Conflict serializable oder nicht?

| schedule $S_A$ |                           |
|----------------|---------------------------|
| $T_1$          | $T_2$                     |
| read(Y, y)     | read(X, x)<br>write(X, x) |
| write(Y, y)    |                           |

Conflict serializable

| schedule $S_C$ |            |
|----------------|------------|
| $T_5$          | $T_6$      |
| read(X, x)     | read(X, x) |
| write(X, x)    |            |

Conflict serializable

| schedule $S_B$ |                           |
|----------------|---------------------------|
| $T_3$          | $T_4$                     |
| read(X, x)     | read(X, x)<br>write(X, x) |
| write(X, x)    |                           |

Nicht conflict serializable

| schedule $S_D$ |             |
|----------------|-------------|
| $T_7$          | $T_8$       |
| read(X, x)     | write(X, x) |
| write(X, x)    |             |

Nicht conflict serializable

# Übersicht

## 2 Schedules und Serialisierbarkeit

- Schedules
- Conflict Serializability
- Conflict Graphs (Precedence Graphs)
- Recoverable Schedules und Cascadeless Schedules

## Conflict Graph (Konfliktgraph)

Erzeugen eines gerichteten Graphen (Conflict Graph oder Precedence Graph) für einen Schedule

Annahme: Eine Transaktion führt auf ein Datenobjekt immer ein Read aus, bevor auf das gleiche Objekt ein Write ausgeführt wird.

## Conflict Graph (Konfliktgraph)

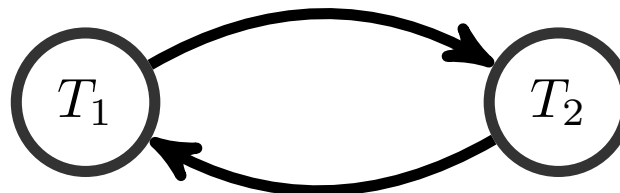
Erzeugen eines gerichteten Graphen (Conflict Graph oder Precedence Graph) für einen Schedule

Annahme: Eine Transaktion führt auf ein Datenobjekt immer ein Read aus, bevor auf das gleiche Objekt ein Write ausgeführt wird.

Gegeben ist ein Schedule für die Transaktionen  $T_1, T_2, \dots, T_n$

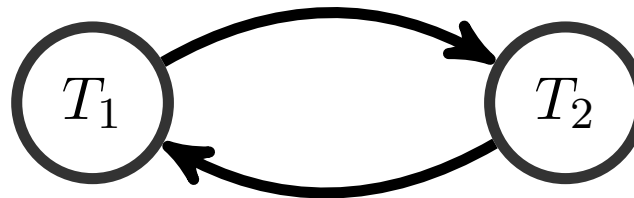
- Die Knoten des Conflict Graphs sind die Transaktions-Ids.
- Eine Kante von  $T_i$  nach  $T_j$  zeigt einen Konflikt zwischen  $T_i$  und  $T_j$  an, wobei  $T_i$  den relevanten Zugriff früher durchführt.
- Manchmal werden Kantenlabels mit dem Namen des involvierten Datenobjekts annotiert.

Beispiel für einen Conflict Graph des Schedules  $S_1$



# Serialisierbarkeit feststellen

Gegeben ist ein Schedule S und ein Conflict Graph, we können wir feststellen, ob der Schedule conflict serializable ist?



## Serialisierbarkeit feststellen

Gegeben ist ein Schedule  $S$  und ein Conflict Graph, we können wir feststellen, ob der Schedule conflict serializable ist?

- Ein Schedule ist **conflict serializable**, wenn der Conflict Graph **acyklisch** ist.
- Intuitiv: Ein Konflikt zwischen zwei Transaktionen erzwingt eine bestimmte Ausführungsreihenfolge (topologische Sortierung)

## Serialisierbarkeit feststellen

Gegeben ist ein Schedule  $S$  und ein Conflict Graph, we können wir feststellen, ob der Schedule conflict serializable ist?

- Ein Schedule ist **conflict serializable**, wenn der Conflict Graph **acyklisch** ist.
- Intuitiv: Ein Konflikt zwischen zwei Transaktionen erzwingt eine bestimmte Ausführungsreihenfolge (topologische Sortierung)

Wir verwenden conflict serializability (und keine andere Definition der Serialisierbarkeit), weil es eine praktikable Implementierung gibt.

# Beispiel: Conflict Graph

schedule  $S_6$

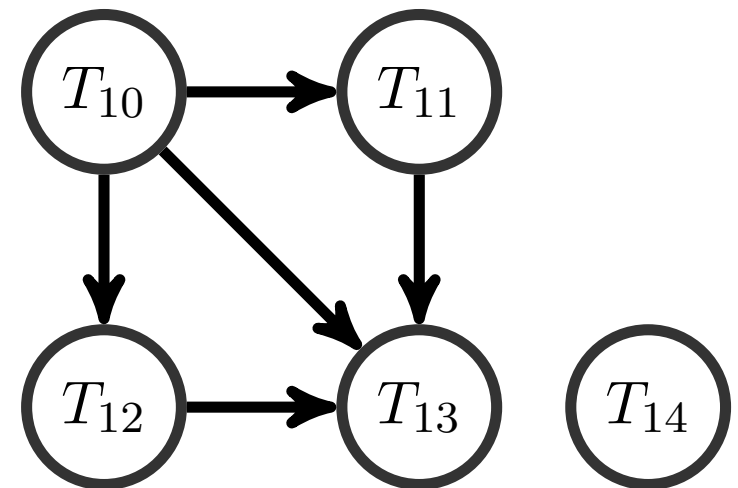
| $T_{10}$                 | $T_{11}$                  | $T_{12}$                  | $T_{13}$                                               | $T_{14}$                                |
|--------------------------|---------------------------|---------------------------|--------------------------------------------------------|-----------------------------------------|
| read(Y, y)<br>read(Z, z) | read(X, x)                |                           |                                                        | read(V, v)<br>read(W, w)<br>write(W, w) |
|                          | read(Y, y)<br>write(Y, y) | read(Z, z)<br>write(Z, z) |                                                        |                                         |
| read(T, t)               |                           |                           | read(Y, y)<br>write(Y, y)<br>read(Z, z)<br>write(Z, z) |                                         |
| read(U, u)               |                           |                           |                                                        |                                         |



# Beispiel: Conflict Graph

schedule  $S_6$

| $T_{10}$                 | $T_{11}$                  | $T_{12}$                  | $T_{13}$                                               | $T_{14}$                                |
|--------------------------|---------------------------|---------------------------|--------------------------------------------------------|-----------------------------------------|
| read(Y, y)<br>read(Z, z) | read(X, x)                |                           |                                                        | read(V, v)<br>read(W, w)<br>write(W, w) |
|                          | read(Y, y)<br>write(Y, y) |                           |                                                        |                                         |
| read(T, t)               |                           | read(Z, z)<br>write(Z, z) |                                                        |                                         |
|                          |                           |                           | read(Y, y)<br>write(Y, y)<br>read(Z, z)<br>write(Z, z) |                                         |
| read(U, u)               |                           |                           |                                                        |                                         |

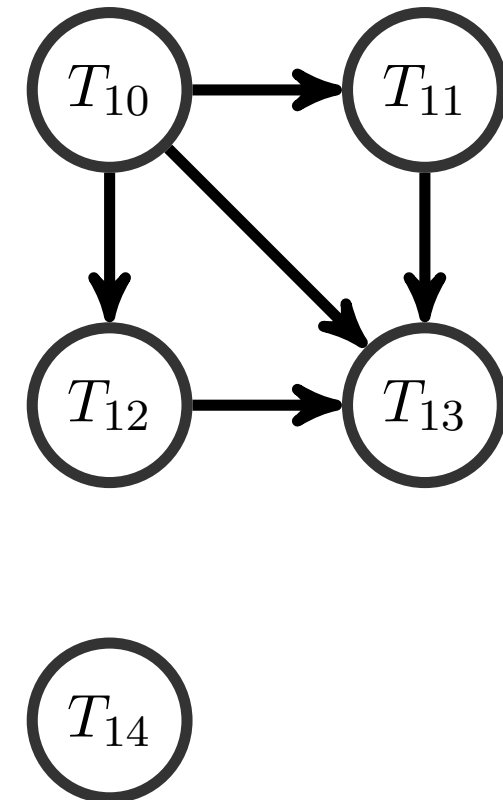


# Menti

Frage 16

# Beispiel Conflict Graph

| schedule $S_6$           |                           |                           |                                                        |                                         |
|--------------------------|---------------------------|---------------------------|--------------------------------------------------------|-----------------------------------------|
| $T_{10}$                 | $T_{11}$                  | $T_{12}$                  | $T_{13}$                                               | $T_{14}$                                |
| read(Y, y)<br>read(Z, z) | read(X, x)                |                           |                                                        | read(V, v)<br>read(W, w)<br>write(W, w) |
|                          | read(Y, y)<br>write(Y, y) | read(Z, z)<br>write(Z, z) |                                                        |                                         |
| read(T, t)               |                           |                           | read(Y, y)<br>write(Y, y)<br>read(Z, z)<br>write(Z, z) |                                         |
| read(U, u)               |                           |                           |                                                        |                                         |



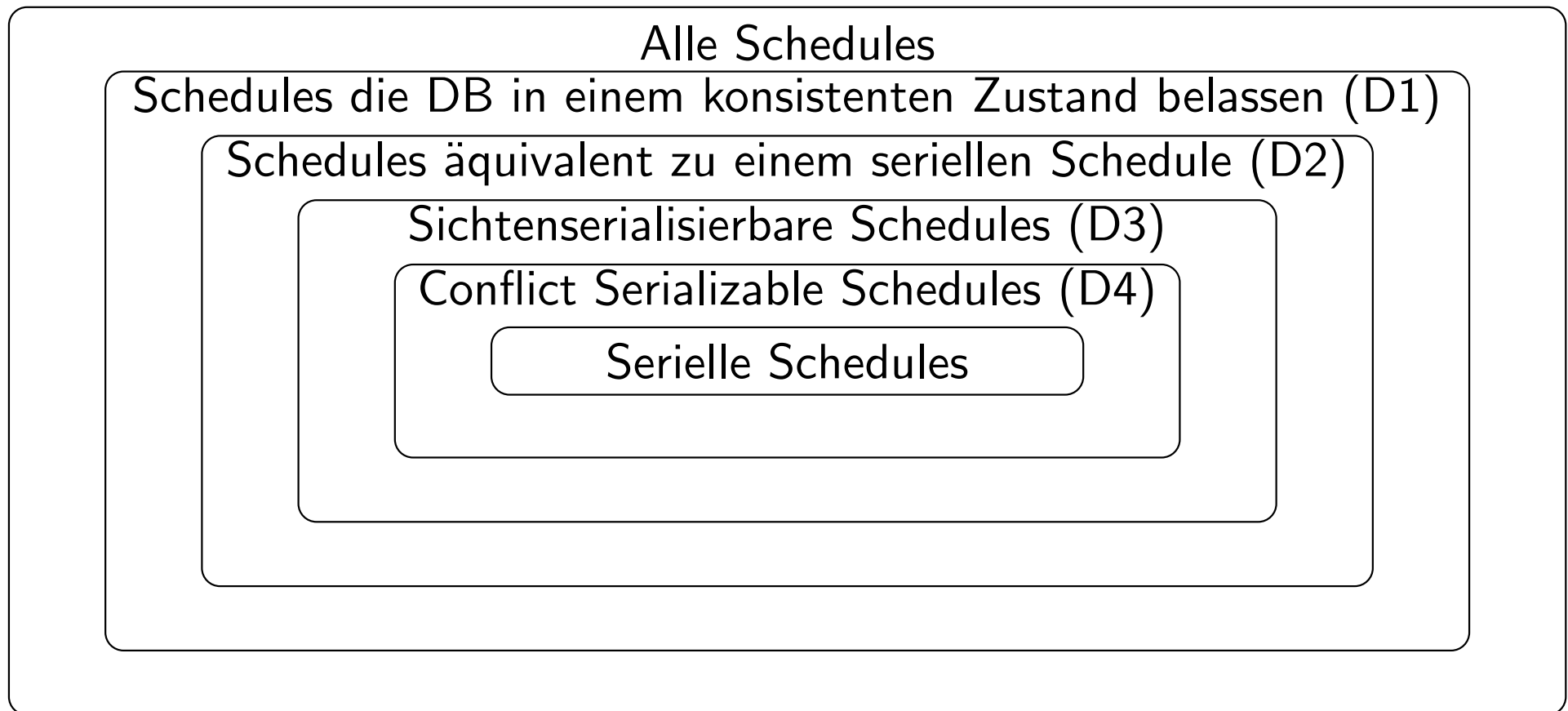
Welche der folgenden Schedules sind konfliktäquivalente serielle Schedules?

$T_{10}, T_{11}, T_{12}, T_{13},$  and  $T_{14}$  Yes

$T_{14}, T_{10}, T_{12}, T_{11},$  and  $T_{13}$  Yes

$T_{14}, T_{13}, T_{12}, T_{11},$  and  $T_{10}$  No

# Beziehungen zwischen Schedules



# Übersicht

- 2 Schedules und Serialisierbarkeit
  - Schedules
  - Conflict Serializability
  - Conflict Graphs (Precedence Graphs)
  - Recoverable Schedules und Cascadeless Schedules

## Transaktionen: Isolation und Atomicity

# Transaktionen können fehlschlagen

# Recoverable Schedules

| schedule $S_A$            |                                     |
|---------------------------|-------------------------------------|
| $T_i$                     | $T_j$                               |
| read(X, x)<br>write(X, x) | read(X, x)<br>write(X, x)<br>commit |
| rollback                  |                                     |

- Wenn  $T_i$  fehlschlägt, muss die Transaktion zurückgesetzt werden, um die **Atomicity (Atomarität)** Eigenschaft zu erhalten (siehe Recovery).

# Recoverable Schedules

| schedule $S_A$            |                                     |
|---------------------------|-------------------------------------|
| $T_i$                     | $T_j$                               |
| read(X, x)<br>write(X, x) | read(X, x)<br>write(X, x)<br>commit |
| rollback                  |                                     |

- Wenn  $T_i$  fehlschlägt, muss die Transaktion zurückgesetzt werden, um die **Atomicity (Atomarität)** Eigenschaft zu erhalten (siehe Recovery).
- Wenn eine andere Transaktion  $T_j$  Daten gelesen hat, die von  $T_i$  geschrieben wurden, dann muss auch  $T_j$  zurückgesetzt werden.  
 $\Rightarrow$  DBS muss sicherstellen, dass Schedules recoverable sind.



# Recoverable Schedules

| schedule $S_A$ |                                     |
|----------------|-------------------------------------|
| $T_i$          | $T_j$                               |
| read(X, x)     | read(X, x)<br>write(X, x)<br>commit |
| write(X, x)    |                                     |
| rollback       |                                     |

Es wird hier ein Dirty-Read ausgeführt!

- Wenn  $T_i$  fehlschlägt, muss die Transaktion zurückgesetzt werden, um die **Atomicity (Atomarität)** Eigenschaft zu erhalten (siehe Recovery).
- Wenn eine andere Transaktion  $T_j$  Daten gelesen hat, die von  $T_i$  geschrieben wurden, dann muss auch  $T_j$  zurückgesetzt werden.  
 $\Rightarrow$  DBS muss sicherstellen, dass Schedules recoverable sind.
- Dieser Schedule ist nicht recoverable.

# Recoverable Schedules

Ein Schedule ist **recoverable**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest, die von  $T_i$  geschrieben wurden, dann muss  $T_i$  vor  $T_j$  committet werden.

| schedule $S_A$ |             |
|----------------|-------------|
| $T_i$          | $T_j$       |
| read(X, x)     |             |
| write(X, x)    |             |
| rollback       |             |
|                | read(X, x)  |
|                | write(X, x) |
|                | commit      |

| schedule $S_B$ |             |
|----------------|-------------|
| $T_i$          | $T_j$       |
| read(Y, y)     |             |
|                | read(X, x)  |
| write(Y, y)    |             |
|                | write(X, x) |
| rollback       |             |
|                | commit      |

# Recoverable Schedules

Ein Schedule ist **recoverable**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest, die von  $T_i$  geschrieben wurden, dann muss  $T_i$  vor  $T_j$  committet werden.

| schedule $S_A$ |             |
|----------------|-------------|
| $T_i$          | $T_j$       |
| read(X, x)     |             |
| write(X, x)    |             |
| rollback       |             |
|                | read(X, x)  |
|                | write(X, x) |
|                | commit      |

Menti: Ist dieser Schedule recoverable?

| schedule $S_B$ |             |
|----------------|-------------|
| $T_i$          | $T_j$       |
| read(Y, y)     |             |
|                | read(X, x)  |
| write(Y, y)    |             |
|                | write(X, x) |
| rollback       |             |
|                | commit      |

Menti: Ist dieser Schedule recoverable?

# Menti

Fragen 17-18

# Recoverable (Wiederherstellbare) Schedules

Ein Schedule ist **recoverable**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest die von  $T_i$  beschrieben wurden, dann muss  $T_i$  vor  $T_j$  committet werden.

| schedule $S_A$ |                                     |
|----------------|-------------------------------------|
| $T_i$          | $T_j$                               |
| read(X, x)     | read(X, x)<br>write(X, x)<br>commit |
| write(X, x)    |                                     |
| rollback       |                                     |
|                |                                     |

recoverable

| schedule $S_B$ |                                     |
|----------------|-------------------------------------|
| $T_i$          | $T_j$                               |
| read(Y, y)     | read(X, x)<br>write(X, x)<br>commit |
| write(Y, y)    |                                     |
| rollback       |                                     |
|                |                                     |

recoverable

# Cascading Rollbacks (Kaskadierendes Rücksetzen)

| schedule $S_{11}$                                      |            |                          |
|--------------------------------------------------------|------------|--------------------------|
| $T_{22}$                                               | $T_{23}$   | $T_{24}$                 |
| read(A, a)<br>read(B, b)<br>write(A, a)<br>write(B, b) | read(A, a) | read(A, a)<br>read(B, b) |
| rollback                                               |            |                          |

Was passiert im Fall eines Rollbacks von  $T_{22}$ ?  
Ist dieser Schedule recoverable?

# Cascading Rollbacks (Kaskadierendes Rücksetzen)

| schedule $S_{11}$                                      |            |                          |
|--------------------------------------------------------|------------|--------------------------|
| $T_{22}$                                               | $T_{23}$   | $T_{24}$                 |
| read(A, a)<br>read(B, b)<br>write(A, a)<br>write(B, b) | read(A, a) | read(A, a)<br>read(B, b) |
| rollback                                               |            |                          |

- $T_{22}$  Rollback  $\Rightarrow$  wir müssen auch  $T_{23}$  und  $T_{24}$  zurücksetzen, weil diese "dirty data" lesen. (Cascading Rollback)
- Der Schedule ist nicht cascadeless (kommt nicht ohne kaskadierendes Rücksetzen aus).
- Aber der Schedule ist recoverable.

# Cascadeless Schedules

Ein Schedule ist **cascadeless**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest, die von  $T_i$  geschrieben wurden, dann muss  $T_i$  vor dem Lesezugriff von  $T_j$  bereits committed sein.

| schedule $S_A$                      |                      |                          |
|-------------------------------------|----------------------|--------------------------|
| $T_1$                               | $T_2$                | $T_3$                    |
| read(A, a)<br>write(A, a)<br>commit | read(A, a)<br>commit | read(A, a)<br><br>commit |



# Menti

Frage 19

# Cascadeless Schedules

Ein Schedule ist **cascadeless**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest, die von  $T_i$  geschrieben wurden, dann muss  $T_i$  vor dem Lesezugriff von  $T_j$  bereits committed sein.

| schedule $S_{11'}$                                                 |                      |                                    |
|--------------------------------------------------------------------|----------------------|------------------------------------|
| $T_{22}$                                                           | $T_{23}$             | $T_{24}$                           |
| read(A, a)<br>read(B, b)<br>write(A, a)<br>write(B, b)<br>rollback | read(A, a)<br>commit | read(A, a)<br>read(B, b)<br>commit |

Dieser Schedule ist  
auch recoverable

# Cascadeless Schedules

Ein Schedule ist **cascadeless**, wenn für jedes Transaktionspaar  $T_i$  und  $T_j$  gilt: Wenn  $T_j$  Daten liest, die von  $T_i$  geschrieben wurden, dann muss  $T_i$  vor dem Lesezugriff von  $T_j$  bereits committed sein.

| schedule $S_{11}'$                                                 |                      |                                    |
|--------------------------------------------------------------------|----------------------|------------------------------------|
| $T_{22}$                                                           | $T_{23}$             | $T_{24}$                           |
| read(A, a)<br>read(B, b)<br>write(A, a)<br>write(B, b)<br>rollback | read(A, a)<br>commit | read(A, a)<br>read(B, b)<br>commit |

Dieser Schedule ist auch recoverable

Cascading Rollbacks können verhindert werden, indem nur von Transaktionen gelesen wird, die bereits committed sind.

# Cascadeless Schedules

- Jeder cascadeless Schedule ist auch recoverable.
- Cascading Rollbacks können schnell zeitaufwändig werden.
- Es ist sinnvoll, sich auf Schedules zu beschränken, die cascadeless sind.

# Zusammenfassung: Transaktionen und Schedules

- Jede Transaktion erhält den konsistenten Zustand der Datenbank.
- Die serielle Ausführung einer Menge von Transaktionen erhält einen konsistenten Zustand.
- Bei einer nebenläufigen Ausführung, können die einzelnen Ausführungsschritte einer Menge an Transaktionen verzahnt sein.
- Ein nebenläufiger Schedule ist serialisierbar, wenn dieser äquivalent zu einem seriellen Schedule ist.
  - Conflict Serializability  
Bevorzugte Methode, da es praktikable Implementierungsmöglichkeiten gibt:  
Conflict Graphs
- Schedules müssen recoverable und sollten cascadeless sein.